

Module 4 Lab Session 3: API Surface & Production Readiness

Field	Value
Module	M4 ASP.NET Core 10 Fundamentals
Exercises	5 (Controllers + CRUD), 6 (ProblemDetails), 7 (Dev/Prod Toggle); optional instructor-led integration exercise after Lab Verification
After this session you can show	CRUD on /api/enrollments, ProblemDetails errors, Dev vs Prod surface (Scalar vs hidden); items 1–11 on the Lab Verification list green

Prerequisites Check Before You Start

Your TmsApi project must contain:

- The pipeline and Correlation ID middleware from Session 1 pipeline order correct, X-Correlation-Id on every response
- EnrollmentService.cs from Session 2 with structured logging and proper log levels (Information / Warning) on all four methods
- EnrollmentWorker injecting IServiceScopeFactory, not IEnrollmentService directly
- PaymentOptions registered with BindConfiguration("Payments") and ValidateOnStart() so missing config crashes at startup
- builder.Services.AddControllers(); in the **builder** section of Program.cs
- app.MapControllers(); in the **middleware** section (after UseAuthentication / UseAuthorization, before app.Run())

If any of these are missing, go back. Session 3 wires the public-facing API on top of all of them; the closing Session 3 exercise your instructor runs will assume this foundation is solid.

What This Session Is Really About

So far the API runs but has no public surface. You have a pipeline, services, configuration, and logging but a frontend cannot do anything with that yet. Session 3 builds three things in that order:

1. **The surface itself** (Exercise 5) controllers that expose /api/enrollments with the four HTTP verbs every TMS client needs, returning the right status codes and the Location header REST clients expect

2. **The error envelope** (Exercise 6) a single, structured JSON shape (RFC 9457 ProblemDetails) that every error response uses, so the Angular frontend in M8 can write one error handler instead of one per failure mode

The environment posture (Exercise 7) Scalar API explorer in Development, hidden in Production; stack traces hidden in Production, replaced by ProblemDetails; the same code, two different surfaces depending on where it runs.

Exercise 5: The Enrollment API (Controllers with Real CRUD)

Context: Now that the infrastructure is stable, you must expose HTTP endpoints for the Angular frontend. The TMS uses MVC Controllers each resource gets its own controller class with a constructor-injected service.

Before you code

1. Confirm `builder.Services.AddControllers();` is in the **builder** section of `Program.cs` (added in Session 2 prerequisites).
2. Confirm `app.MapControllers();` is in the **middleware** section (after auth, before `app.Run();`).
3. Create a Controllers folder in your project.

Part A: GET Endpoints

Create a new file called `Controllers/EnrollmentsController.cs`:

using `Microsoft.AspNetCore.Mvc;`

```
[ApiController]
[Route("api/enrollments")]
public class EnrollmentsController(IEnrollmentService enrollmentService) : ControllerBase
{
    // GET /api/enrollments returns all enrollment records
    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var enrollments = await enrollmentService.GetAllAsync();
        return Ok(enrollments);
    }

    // GET /api/enrollments/{id} returns one or 404
    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(string id)
    {
        var record = await enrollmentService.GetByIdAsync(id);
        return record is not null ? Ok(record) : NotFound();
    }
}
```

```
}  
}
```

Test it: dotnet run, then call GET /api/enrollments. You should see 200 OK with an empty array (no enrollments yet).

Part B: POST with 201 + Location

Add a POST action to the same controller:

```
// POST /api/enrollments creates and returns 201 with Location header  
[HttpPost]  
public async Task<IActionResult> Create([FromBody] CreateEnrollmentRequest request)  
{  
    var record = await enrollmentService.EnrollAsync(request.StudentId, request.CourseCode);  
    return CreatedAtAction(nameof(GetById), new { id = record.Id }, record);  
}
```

Add the request model at the bottom of the same file (or a separate Models folder):

```
public record CreateEnrollmentRequest(string StudentId, string CourseCode);
```

CreatedAtAction does three things: sets the status to **201**, sets the Location header to the URL of the new resource (/api/enrollments/{id}), and returns the created record in the body. This is the correct HTTP semantic for resource creation not a bare 200 OK.

Part C: DELETE with 204 / 404

Add a DELETE action:

```
// DELETE /api/enrollments/{id} returns 204 or 404  
[HttpDelete("{id}")]  
public async Task<IActionResult> Delete(string id)  
{  
    var deleted = await enrollmentService.DeleteAsync(id);  
    return deleted ? NoContent() : NotFound();  
}
```

NoContent() returns **204** the standard response for a successful delete with no body.

Run / Call / Expected / Common failure

- **Run:** dotnet run
- **Call:** Use curl or your browser/Scalar to test all four endpoints:

1. GET all (empty list)

curl http://localhost:5000/api/enrollments

Expected: 200 with []

2. POST a new enrollment

curl -X POST http://localhost:5000/api/enrollments \

```
-H "Content-Type: application/json" \  
-d '{"studentId":"S-001","courseCode":"CS-101"}'  
# Expected: 201 Created with Location header and body
```

3. GET the created enrollment (use the id from step 2)
curl http://localhost:5000/api/enrollments/{id}
Expected: 200 with the record

4. DELETE the enrollment
curl -X DELETE http://localhost:5000/api/enrollments/{id}
Expected: 204 No Content

5. GET the deleted enrollment
curl http://localhost:5000/api/enrollments/{id}
Expected: 404 Not Found

- **Common failure 404 for everything:** Missing AddControllers() or MapControllers() in Program.cs, or wrong [Route] prefix.
- **Common failure POST returns 200 instead of 201:** You used Ok() instead of CreatedAtAction(). The Location header is required for REST compliance.
- **Common failure POST returns 500 with “Value cannot be null”:** The [FromBody] attribute is missing, or the Content-Type: application/json header is missing from the request.

Troubleshooting

- If CreatedAtAction throws “No route matches the supplied values,” the nameof(GetById) argument does not match your GET action’s name. They must be identical.
- If curl is not available on Windows, use PowerShell: Invoke-WebRequest -Uri "http://localhost:5000/api/enrollments" -Method GET

Exercise 6: The Consistent Fault (Standardized Error Handling)

Context: When the API crashes, it currently returns a raw HTML error page. The TMS frontend needs structured JSON (**RFC 9457 ProblemDetails**) so it can show a helpful alert whether the failure was saving an **Enrollment**, publishing a **Course** outline, recording an **Assessment** attempt, or issuing a **Certificate**.

UseStatusCodePages is optional but recommended alongside ProblemDetails it turns **empty-body** status codes (such as bare **404** responses) into the same consistent JSON shape.

Implementation: ProblemDetails

Implement the standard **ProblemDetails** exception handler in Program.cs.

```
// TODO 1: In the Builder section, add the ProblemDetails service
// builder.Services.AddProblemDetails();
```

```
// TODO 2: In the Middleware section, use the Exception Handler
// app.UseExceptionHandler();
```

```
// TODO 3 (optional alignment with Essentials): app.UseStatusCodePages();
```

```
// TODO 4: Map a test route '/api/error' that intentionally throws.
```

You need an exception to throw. Add this class to your project (a new file or at the bottom of EnrollmentService.cs):

```
public class TmsDatabaseException(string message) : Exception(message);
```

Then wire the test route in Program.cs:

```
app.MapGet("/api/error", () =>
{
    throw new TmsDatabaseException("Simulated database failure for ProblemDetails testing");
});
```

Run / Call / Expected / Common failure

- **Run:** dotnet run
- **Call:** GET http://localhost:5000/api/error (or whatever path throws inside your test route).
- **Expected:** Response is **JSON** with at least type, title, status, and detail fields (RFC 9457 ProblemDetails shape), not a stack trace as HTML.
- **Common failure raw HTML:** UseExceptionHandler() is missing or registered **after** endpoint execution so exceptions never reach it. Place it **before** routing/mapping per your Essentials sample.

Exercise 7: The Environment Toggle (Dev vs Prod)

Context: In Development, you want rich diagnostics and an interactive API explorer. In production, you must **hide** explorers and avoid leaking stack traces.

Before you write any code

1. Install Scalar (required for MapScalarApiReference):
dotnet add package Scalar.AspNetCore
2. Add the OpenAPI service and the Scalar using to Program.cs:
using Scalar.AspNetCore;

```
var builder = WebApplication.CreateBuilder(args);
```

```
// ... your existing service registrations ...  
builder.Services.AddOpenApi(); // Required before MapOpenApi() will work
```

Without AddOpenApi(), calling MapOpenApi() later will throw at startup.

Your Task: Environment-Aware Configuration

```
// TODO 1: Check if the app is running in Development mode.  
// Stuck? if (app.Environment.IsDevelopment()) { ... }  
  
// TODO 2: In Development only expose the OpenAPI document and an interactive API explorer.  
// Use the built-in MapOpenApi() and MapScalarApiReference().  
// Stuck? app.MapOpenApi(); app.MapScalarApiReference();  
  
// TODO 3: In Production use the exception handler middleware so stack traces  
// are never shown to external users.  
// Stuck? app.UseExceptionHandler();  
  
// TODO 4: Run in both environments and verify:  
// - In Development: can you browse /scalar/v1 and see your endpoints?  
// - In Production: does a thrown exception return ProblemDetails JSON, not a stack trace?
```

Run / Call / Expected / Common failure

Development

- **Run:** dotnet run (default is Development in launchSettings.json).
- **Call:** Browser <http://localhost:5000/scalar/v1> (path may vary slightly by template; use the link shown if different).
- **Expected:** Scalar UI loads and lists your OpenAPI document.

Production (Windows PowerShell)

```
$env:ASPNETCORE_ENVIRONMENT = "Production"  
dotnet run
```

- **Call:** Same Scalar URL as above, and hit an endpoint that throws (or /api/error if you kept it).
- **Expected:** /scalar/v1 returns **404** or is otherwise **not** exposed; errors return **ProblemDetails JSON**, not HTML stack traces.

Reset terminal when done:

```
Remove-Item Env:ASPNETCORE_ENVIRONMENT
```

- **Common failure Scalar does not compile:** Package not added (dotnet add package Scalar.AspNetCore) or missing using Scalar.AspNetCore;.

Session Checkpoint

Before completing the module, confirm all five:

- ☐ POST /api/enrollments with {"studentId":"S-001","courseCode":"CS-101"} returns 201 Created with a Location header pointing to /api/enrollments/{id} verify the header with curl -i
- ☐ DELETE /api/enrollments/{id} returns 204 No Content for an existing record and 404 Not Found for a non-existent one
- ☐ GET /api/error returns RFC 9457 ProblemDetails JSON (type, title, status, detail) not an HTML stack trace
- ☐ In Development, /scalar/v1 loads the API explorer; in Production, it returns 404
- ☐ In Production, hitting /api/error returns the same ProblemDetails JSON envelope not a stack trace, not raw HTML

If any item fails, finish it before your instructor runs the closing Session 3 exercise (if scheduled).

Lab Verification Full Module

This list is split on purpose:

- **Items 1–11:** Self-check your TMS API through Session 3. Fix failures before any instructor-led integration exercise.
- **Item 12:** After that exercise (if your programme runs it), confirm the new behaviour at the HTTP boundary and that items 1–11 still pass.

Items 1–11

Run your API using dotnet run and verify each row:

#	Check	Session	Tier
1	GET /api/assessments/results without credentials returns 401 Unauthorized	1	T1
2	Every response includes an X-Correlation-Id header (check with curl -v)	1	T1
3	Console logs show structured lines with [CorrelationId], method, path, status code, elapsed time	1	T1
4	With ValidateScopes = true, injecting IEnrollmentService directly into a singleton throws at startup	2	T2
5	After the IServiceScopeFactory fix, the app starts and 15 concurrent calls do not cross-contaminate data	2	T2

#	Check	Session	Tier
6	Deleting Payments:GatewayUrl from appsettings.json crashes the app at startup (not at first request)	2	T3
7	Duplicate enrollment logs [Warning]; success logs [Information]; not-found logs [Warning]	2	T4
8	POST /api/enrollments returns 201 Created with a Location header (not bare 200)	3	T5
9	DELETE /api/enrollments/{id} returns 204 ; a second DELETE on the same ID returns 404	3	T5
10	/api/error returns ProblemDetails JSON with type, title, status, detail no HTML	3	T5
11	In Development, /scalar/v1 loads the API explorer; in Production, it returns 404	3	T5

If all eleven pass, you are ready for M4 wrap-up and (if applicable) your instructor's closing exercise.

After the closing exercise 1 item

#	Check	Session	Tier
12	Instructor-assigned integration work is observable at the HTTP boundary; items 1–11 still pass (no regressions)	3	T6

If item 12 passes alongside items 1-11, your backend infrastructure is production-ready.